САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО

ЛАБОРАТОРНАЯ РАБОТА № 3

«Реализация спроектированной системы»

Дисциплина: «Основы разработки безопасных цифровых систем»
Вариант 1: «Заявки на справки и документы»
Особенность: юридическая значимость выдаваемого документа

Выполнил: Высоцкий А. А.
Группа: 5130901/20103

Реестр расхождений с проектом ЛР № 2 (сводно; подробный журнал — раздел 8)

ID	Изменение	Причина и граница
CH-01	Стек API/worker: Python → Go 1.25	Выполнено по прямому требованию; API-контракты сохранены.
CH-02	LLM Gateway/провайдер не развёртывается	AI endpoint авторизован и возвращает 501, как разрешено заданием.
CH-03	Дедупликация уведомлений: новое поле source_outbox_event_id	Добавлены UNIQUE и FK на outbox-событие.
CH-04	Почта HTTPS/JSON → локальный SMTP Mailpit	Контракт асинхронного уведомления сохранён; внешний email не отправляется.
CH-05	Сервисные роли PostgreSQL разделены	API и worker подключаются разными ролями с ограниченными GRANT.
CH-06	Outbox publisher вынесен в worker	Worker опрашивает outbox, ждёт publisher confirmation и обрабатывает очередь.
CH-07	request_id, выдача и байтовый формат уточнены	UUID генерируется сервером; документ читается в Base64; title обязателен.
CH-08	Локальный Compose без TLS	Все опубликованные web/API/UI-порты ограничены localhost; TLS требуется для production.
CH-09	USER: чужой объект → 404; ADMIN вне scope → 403	Анти-enumeration и явное разграничение подразделений.
CH-10	Строгий JSON decoder отклоняет неизвестные поля	Защита от mass assignment; владелец и статус назначаются сервером.
CH-11	request_id генерируется и нормализуется middleware	Корреляция аудита не зависит от клиентского заголовка.
CH-12	document_title обязателен; bytes передаются в Base64	Соответствие контракту и корректный транспорт BYTEA.
CH-13	Outbox publisher/consumer вынесен в worker	Подтверждение публикации и асинхронная обработка уведомлений.
CH-14	Внешние JS/CSS и CSP без unsafe-inline	Работающий веб-клиент при строгой политике содержимого.
CH-15	Idempotency-Key с fingerprint в БД	Повтор POST с тем же ключом/телом возвращает исходную заявку; иной payload даёт 409.
CH-16	Fail-closed сброс пароля и backoff SMTP	Ошибки БД отличимы от 404; сессии отзываются до commit; SMTP retry ждёт 2 с.

Git: локальная история включена в комплект. Удалённый репозиторий не публиковался, поскольку URL/доступ к аккаунту не предоставлены; ссылка должна быть добавлена после публикации.

1. Цель, исходные данные и границы реализации

Цель работы — реализовать и подготовить к локальному развёртыванию систему обработки заявок на университетские справки и документы по проектной документации ЛР № 2. Реализация использует Go 1.25 для REST API и фонового worker, PostgreSQL как источник бизнес-истины, Redis для opaque-сессий/лимитов/кэша, RabbitMQ для асинхронных уведомлений, Mailpit как локальный SMTP-приёмник и веб-клиент на HTML/CSS/JavaScript, отдаваемый Nginx.

Предметная область требует особенно строго обеспечивать целостность выданного документа, корректность статуса, связь документа с заявкой и возможность восстановить историю действий. Из ЛР № 1–2 не следует конкретный формат юридического оформления или способ электронной подписи. Поэтому в проекте документ представляет учебный текстовый артефакт с SHA-256 и доказательной записью выдачи; он не объявляется подписанным или юридически действительным только из-за наличия хеша.

По прямому исключению в задании ЛР № 3 интеграция с языковой моделью не реализована: аутентифицированный маршрут проверки полноты заявки возвращает HTTP 501. Остальные прикладные операции реализованы в пределах контрактов и с оговорёнными в журнале изменениями.

2. Соответствие заданию ЛР № 3

Пункт задания	Артефакт реализации
1. Проект и окружение	Структура по контейнерам: api, worker, web, db, requests, docs; `.env.example`, `.gitignore`, README, Docker Compose.
2. PostgreSQL	Скрипт `db/schema.sql` выполняется при первом запуске volume; отдельные роли `documents_app` и `documents_worker`; seed запускается приложением.
3. Аутентификация	Argon2id с параметрами ЛР № 2 и pepper из окружения; opaque bearer с Redis TTL; logout отзывает token; одноразовое восстановление доступа.
4. Операции API	Все операции OP-01…OP-07 и OP-09 реализованы; OP-08 исключена по условию ЛР № 3; строгий decoder, параметры SQL и нейтральные ошибки.
5. Межсервисные связи	Redis, PostgreSQL, RabbitMQ, worker и SMTP-приёмник реально включены в Compose; Outbox и publisher confirmations.
6. Клиент	Веб-интерфейс позволяет вход/восстановление, создать и просмотреть заявки, сменить статус, выдать и прочитать документ, открыть аудит.
7. Проверки и карта кода	`requests/smoke.sh`, unit-тесты, `docs/code-map.md`, журнал изменений и реестр статической проверки.

3. Фактическая архитектура и компоненты

Рисунок ниже показывает реально заданные в `docker-compose.yml` контейнеры и транспорт между ними. Диаграммы исходного проекта ЛР № 2 сохранены в `docs/reference_l2/`, но фактическая схема реализации дана отдельно: LLM отсутствует, почтовая граница заканчивается в Mailpit, а Outbox публикуется фоновым worker.

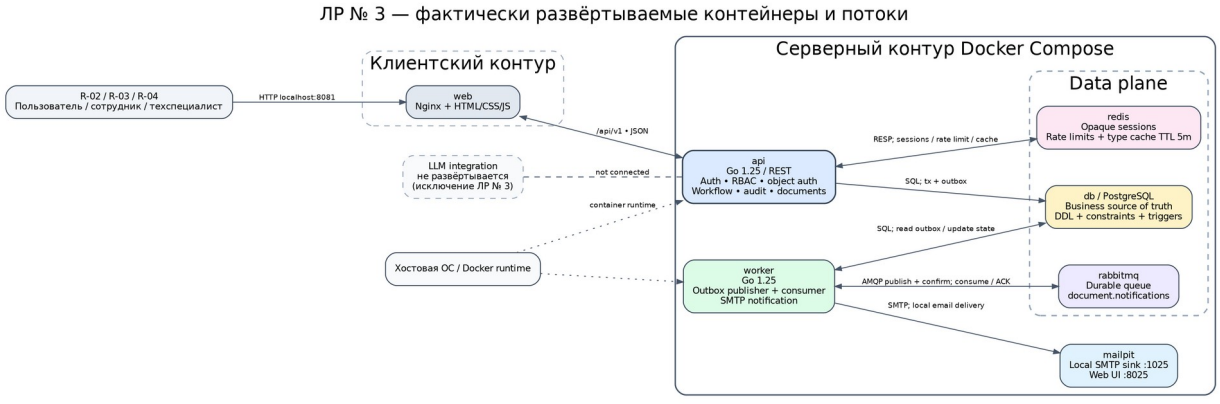


Рисунок 1 — Развёртываемая архитектура ЛР № 3 и фактические межсервисные потоки

Контейнер	Назначение	Архитектурное решение
web	Nginx + HTML/CSS/JS	Порт 8081 на localhost; только HTTP API вызывает backend; прямого соединения с PostgreSQL нет.
api	Go 1.25 REST API	Порт 8080 на localhost; auth/RBAC/object authorization, прикладные транзакции, аудит, Redis и PostgreSQL.
db	PostgreSQL 17	Не публикует порт на хост; схема и ограничения создаются скриптом инициализации.
redis	Redis 7	Opaque-сессии с TTL 1800 с, fixed-window rate limiting, кэш справочника типов на 5 минут.
rabbitmq	RabbitMQ 4 management	Durable queue `document.notifications`; опубликован интерфейс управления 15672 только на localhost.
worker	Go 1.25 worker	Читает outbox, публикует durable сообщения с подтверждением, consume/ACK, SMTP в Mailpit.
mailpit	Mailpit SMTP sink	SMTP внутри Compose на 1025, UI для проверки писем на localhost:8025.

Все компоненты запускаются одной командой `docker compose up --build -d`. Схема PostgreSQL выполняется автоматически только на пустом volume. Запуск и подробности остановки/сброса описаны в README.

4. Модель данных и учётные записи

DDL сохранён на основе ЛР № 2. В схеме 13 таблиц: `roles`, `users`, `user_roles`, `document_types`, `applications`, `application_status_history`, `documents`, `document_issue_events`, `audit_events`, `ai_checks`, `notifications`, `outbox_events`, `password_reset_tokens`. Сохраняются обязательный `owner_user_id`, FK/CHECK/UNIQUE, индексы по владельцу и подразделению, неизменяемость выданного документа и append-only защита аудита/событий выдачи. Дополнительно введён `Idempotency-Key` с fingerprint тела для безопасного повтора POST (CH-15).

Единственное структурное добавление в DDL для ЛР № 3 — `notifications.source_outbox_event_id UUID UNIQUE REFERENCES outbox_events(outbox_event_id)`. Оно связывает notification с исходным outbox-событием и позволяет не создавать вторую строку уведомления при повторном сканировании outbox. Изменение перечислено в журнале CH-03.

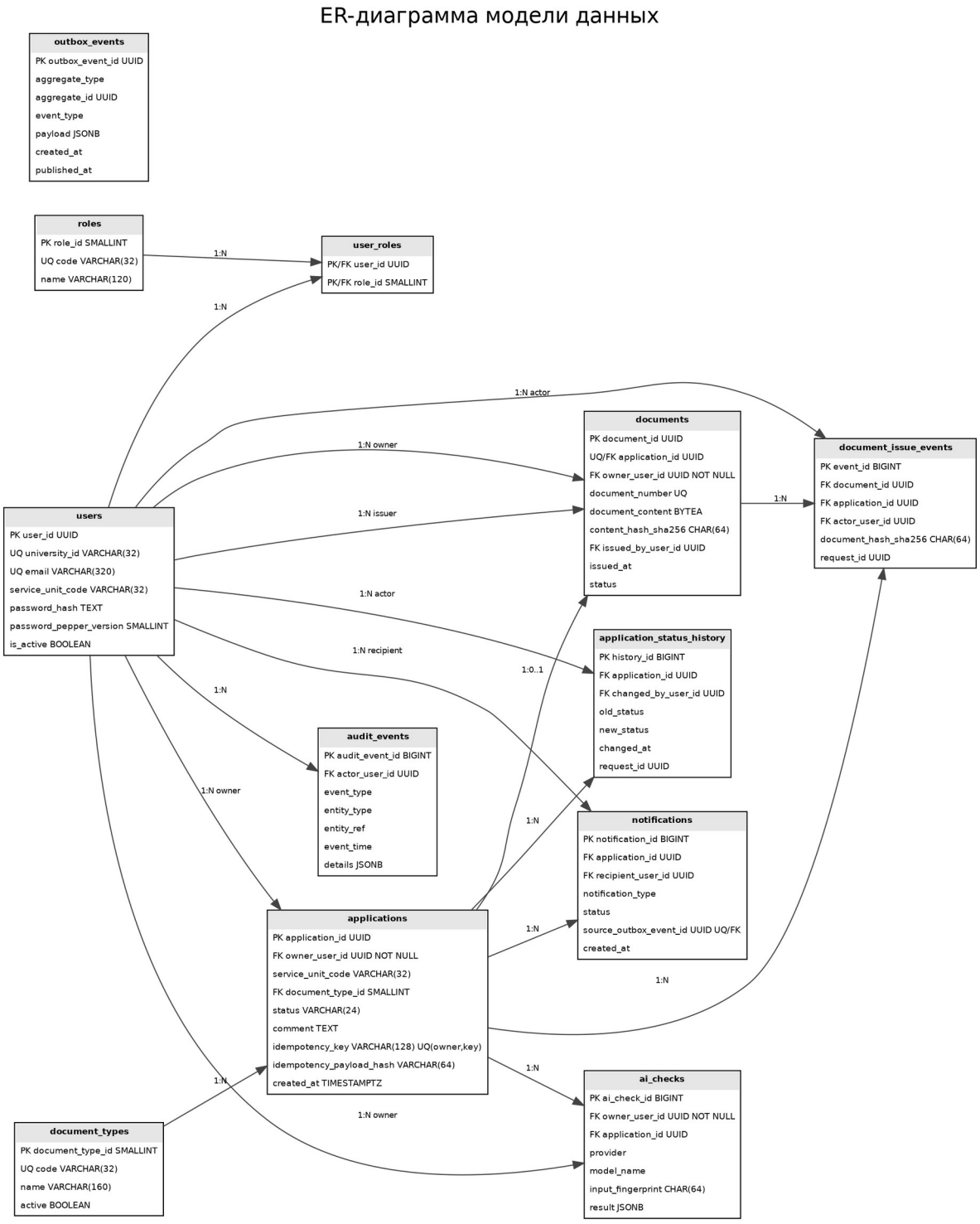


Рисунок 2 — ER-модель PostgreSQL с ключом дедупликации Outbox

Сервис API подключается как `documents_app`, worker — как `documents_worker`; роли и пароли берутся из переменных окружения. Скрипты GRANT не дают API права DDL, DELETE или DROP, а worker не имеет права менять users/applications/documents. Это уменьшает влияние компрометации отдельного процесса.

Seed создаёт как минимум двух пользователей USER с разными заявками, одного ADMIN и одного TECH. Пароли не сохраняются открытым текстом. На первом старте API формирует verifier Argon2id с солью и pepper, полученным из `PASSWORD_PEPPER`; соль, параметры и результат входят в verifier, сам pepper в базе отсутствует.

5. Реализация контрактов API

ID	Метод / маршрут	Ответы (основные)	Проверка/инвариант
OP-01	POST `/api/v1/auth/login`	200 / 400 / 401 / 429 / 500	Opaque bearer; одинаковый текст ошибки для неизвестного логина и неверного пароля.
OP-02	GET `/api/v1/applications/{application_id}`	200 / 401 / 403 / 404 / 429 / 500	USER — owner predicate и 404 для чужой заявки; ADMIN — scope подразделения.
OP-03	POST `/api/v1/applications`	201 / 400 / 401 / 422 / 429 / 500	Owner берётся из серверной сессии; status=`submitted`; неизвестные поля отклоняются.
OP-04	PATCH `/api/v1/applications/{application_id}/status`	200 / 400 / 401 / 403 / 404 / 409 / 422 / 500	ADMIN, scope и конечный автомат; reason нужен для rejected/cancelled; транзакция+history+audit+outbox.
OP-05	GET `/api/v1/applications`	200 / 400 / 401 / 403 / 429 / 500	status/type/date filters, page/page_size и серверный предикат владельца или scope.
OP-06	POST `/api/v1/applications/{application_id}/issue`	201 / 400 / 401 / 403 / 404 / 409 / 422 / 500	Только approved и scope ADMIN; row lock; UNIQUE по application_id; документ, история, audit и outbox атомарны.
OP-07	GET `/api/v1/documents/{document_id}`	200 / 401 / 403 / 404 / 429 / 500	Проверка owner/scope; содержимое в Base64; хеш SHA-256 отдельно.
OP-08	POST `/api/v1/applications/{id}/ai/completeness-check`	501	Не реализуется по прямому исключению ЛР № 3; маршрут требует сессию.
OP-09	GET `/api/v1/audit/events`	200 / 401 / 403 / 500	ADMIN получает бизнес-аудит, TECH — только ограниченные технические события.

Дополнительные маршруты, необходимые для полного сценария аутентификации: `POST /api/v1/auth/logout`, `POST /api/v1/auth/recovery`, `POST /api/v1/auth/recovery/confirm`. Сброс пароля использует одноразовый код с TTL 20 минут; в PostgreSQL хранится его SHA-256, а в локальном стенде код выдаётся через журнал API как разрешённая заданием заглушка.

6. Реализация ключевых мер безопасности

Контроль	Реализация
Аутентификация	Токен — случайные 32 байта в hex (256 бит); Redis хранит `session:<token>` с TTL. Роль и subject получают из серверной сессии, а не из тела запроса.
Парольный verifier	Argon2id: m=65536 KiB, t=3, p=2, key=32 bytes; соль 16 случайных байт; pepper только из окружения. Поддельный verifier создаёт дополнительную стоимость для несуществующего логина.
Объектная авторизация	USER получает только собственные заявки и документы; ADMIN — только записи своего `service_unit_code`; TECH не получает бизнес-доступ к выдаче.
Mass assignment	Decoder использует `DisallowUnknownFields`; тело создания не может устанавливать owner, status, role, UUID или timestamps; после валидации поля на INSERT формирует API.
Идемпотентность POST	Необязательный `Idempotency-Key` уникален в пределах владельца. Хеш нормализованного тела предотвращает повторное создание при повторе того же запроса и даёт 409 при попытке переиспользовать ключ для другого запроса.
SQL injection	SQL параметризован через `$1`, `$2` и т. д.; не выполняется конкатенация запроса из пользовательского ввода.
Частота запросов	Redis Lua fixed-window лимиты; если Redis недоступен, операция не пропускается без защиты, а возвращает 503.
Юридически значимая выдача	`SELECT ... FOR UPDATE`, состояние `approved`, scope ADMIN;

Контроль	Реализация
	уникальность выдачи на заявку; запись документа, истории, issue event, audit и outbox в одной PostgreSQL-транзакции.
Неизменяемость	Триггер запрещает менять содержание/существенные реквизиты документа после выдачи и физически удалять документ; аудит и issue events append-only.
Почтовое событие	Outbox фиксируется вместе с бизнес-изменением; worker публикует persistent AMQP event, ждёт confirmation и ставит `published_at` только после подтверждения.
Ошибка/отказ сервиса	Клиенту возвращается нейтральное сообщение; SQL/stack trace не раскрываются. При ошибке БД не маскируются под 404; при недоступном Redis сброс пароля откатывается; SMTP retry задерживается на 2 секунды.

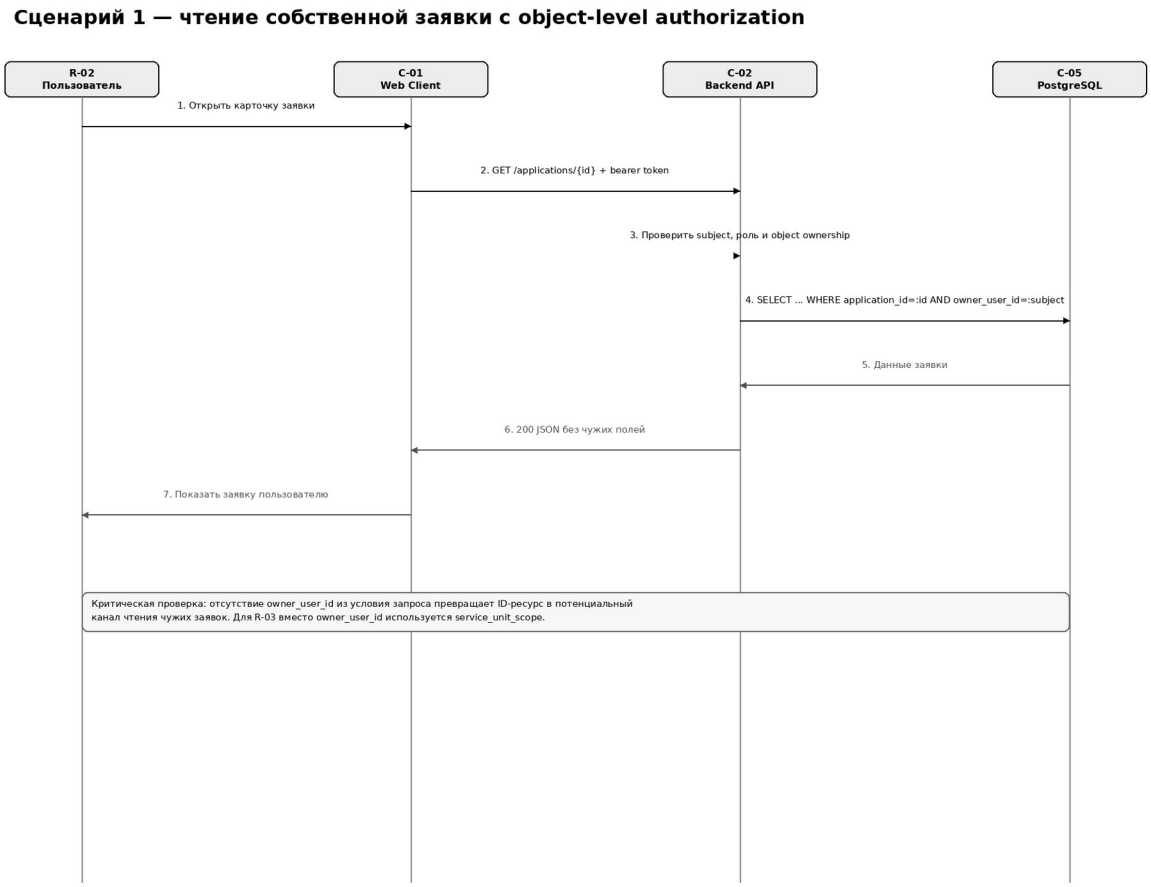


Рисунок 3 — Контроль объектной авторизации: право проверяется на сервере до возврата объекта

7. Ключевые последовательности исполнения

7.1. Чтение собственной заявки

1. Клиент передаёт `Authorization: Bearer ...` и UUID заявки; UUID не считается доказательством владения.
2. Auth middleware получает session record из Redis и извлекает `user_id`, role и scope. При отсутствии/просрочке записи API возвращает 401.
3. API строит параметризованный SQL-запрос. USER получает результат только при совпадении owner_user_id; чужая запись маскируется 404. ADMIN дополнительно ограничен scope подразделения.
4. Ответ формируется из явно разрешённых полей, а не из полной строки таблицы; хеш пароля и внутренние служебные поля не возвращаются.

7.2. Выдача документа

5. API проверяет активную серверную сессию и роль ADMIN, затем декодирует только разрешённое поле `document_title`.
6. Внутри транзакции строка заявки блокируется `FOR UPDATE`. Приложение проверяет подразделение и состояние `approved`; если документ уже выдан, повтор получает 409.
7. API генерирует document_id/document_number/content, вычисляет SHA-256 и атомарно сохраняет документ, переход `approved -> issued`, историю, событие выдачи, аудит и outbox.
8. После commit worker читает событие outbox, создаёт/переиспользует notification по уникальному source_outbox_event_id, публикует сообщение в durable queue и ждёт publisher confirmation.
9. Consumer отправляет SMTP-письмо в Mailpit. При сбое delivery сообщение повторяется; при повторной доставке уже отмеченная как sent notification подтверждается без повторной отправки в рамках этого сценария.

Именно Outbox сохраняет главный инвариант варианта 1: успешная выдача не зависит от доступности почтового сервиса и не откатывается при сбое уведомления. Доставка имеет семантику at-least-once; редкий сбой после фактического SMTP-приёма, но до фиксации статуса, всё ещё может привести к повторному письму.

8. Журнал расхождений с ЛР № 2

ID	Расхождение	Обоснование / контроль
CH-01	Python в документации ЛР № 2 → Go 1.25 для API и worker	Задан пользователем; ЛР № 3 допускает выбор стека. Контракты и архитектурные роли сохранены.
CH-02	LLM Gateway и внешний провайдер исключены	Исключение прямо указано в задании ЛР № 3; endpoint отвечает 501 после auth.
CH-03	`notifications.source_outbox_event_id UUID UNIQUE REFERENCES outbox_events`	Необходим для дедупликации notification на повторной публикации outbox-события.
CH-04	Внешний HTTPS/JSON email заменён SMTP в Mailpit	Разрешённая локальная заглушка; реальная доставка внешнему пользователю не заявлена.
CH-05	Redis используется для сессий, rate limiting и кэша типов документов	Реализует назначение Redis из ЛР № 2; TTL кэша 5 минут.
CH-06	Раздельные PostgreSQL-роли для API и worker	Принцип наименьших привилегий; worker не меняет users/applications/documents.
CH-07	Код восстановления печатается в журнал API	Локальная заглушка в соответствии с заданием; только для учебного стенда, не для production.
CH-08	TLS внутри Compose не настроен	Упрощение локального стенда; опубликованные порты ограничены localhost; production требует TLS.
CH-09	Чужой объект скрывается от USER как 404; scope-отказ ADMIN — 403	Сохраняет anti-enumeration для пользователя и явный контроль границы подразделения.
CH-10	Неизвестные JSON-поля строго отклоняются с 400	Усиление против mass assignment вместо игнорирования полей; валидный create отдельно показывает серверные owner/status.
CH-11	UUID request_id генерируется сервером при отсутствии/невалидности клиентского значения	Исключает ошибку выдачи из-за отсутствующего заголовка и обеспечивает корреляцию.
CH-12	`document_title` обязателен; байтовый контент возвращается Base64	Сохранён контракт названия; Base64 — корректное транспортное представление BYTEA.
CH-13	Outbox publisher/consumer размещён в worker	Сохраняет транзакционность outbox; применены publisher confirmation и persistent queue; фактический поток показан на рисунке 1.
CH-14	Inline JS/CSS заменены внешними ресурсами, введена строгая CSP для Nginx и API	Клиент получает работоспособную CSP без `unsafe-inline`; inline onclick заменены обработчиками data-action.
CH-15	Idempotency-Key с fingerprint в БД	Повтор POST с тем же ключом/телом возвращает исходную заявку; другой payload даёт 409.
CH-16	Fail-closed сброс пароля и backoff SMTP	Ошибки БД отличимы от 404; сессии отзываются до commit; SMTP retry ждёт 2 с.

9. Запуск и пользовательские сценарии

Для запуска требуется Docker Engine с Docker Compose v2 и доступ к registry при первом скачивании образов и Go-модулей. Go на хосте не нужен: многостадийные Dockerfile используют Go 1.25. На странице README перечислены тестовые роли; пароли относятся исключительно к локальному стенду. Измените демонстрационные секреты в `.env` до запуска.

10. Скопировать `.env.example` в `.env` и заполнить отдельные пароли PostgreSQL, API role, worker role, Redis, RabbitMQ и pepper длиной не менее 32 байт.
11. В корне проекта выполнить `docker compose up --build -d`, затем `docker compose ps` и `curl -i http://localhost:8080/healthz`.
12. Открыть Web UI `http://localhost:8081`, Mailpit `http://localhost:8025` и RabbitMQ Management `http://localhost:15672`.
13. Для сброса локальных данных выполнить `docker compose down -v`, затем повторить запуск. Команда удаляет данные локального volume.
14. Для демонстрации проверок выполнить `./requests/smoke.sh`. Скрипт использует `curl`, `jq`, Docker Compose и `.env`.

Демонстрационные учётные записи: `student01` и `student02` (роль USER), `staff01` (ADMIN/R-03), `tech01` (TECH/R-04). Аутентификация и авторизация выполняются backend, а скрытие кнопки в клиенте не используется как контроль безопасности.

10. Проверка проекта и ограничения фактической верификации

В доступной среде выполнены статические проверки: форматирование исходников `gofmt`, синтаксис shell-скриптов `bash -n`, разбор `docker-compose.yml` парсером YAML, структурные проверки 13 таблиц PostgreSQL/триггеров/ключа outbox, генерация PNG диаграмм Graphviz. В `api/main_test.go` добавлены unit-тесты проверки UUID, строгого JSON decoder и verifier Argon2id.

Важно: здесь установлен Go 1.23.2 вместо требуемого Go 1.25, попытка загрузить toolchain 1.25 остановилась из-за DNS/сети. Docker CLI в среде подготовки также отсутствует. Поэтому компиляция под Go 1.25, фактический `docker compose up`, `go test` и end-to-end smoke-скрипт здесь не исполнялись. В отчёте они не представлены как успешно выполненные. Полные ожидаемые коды ответов и последовательность воспроизводимой проверки записаны в `docs/validation.md` и `requests/smoke.sh`; повторить их нужно на целевом компьютере.

Проверка	Результат
gofmt для api/worker	Выполнено
`bash -n` smoke/init scripts	Успешно
Разбор Compose YAML и наличие 7 services	Успешно статически
Структура DDL: 13 tables, owner fields, outbox FK/UNIQUE, protective triggers	Успешно статически
`go test ./...` при Go 1.25	Не исполнялся: скачивание toolchain недоступно
`docker compose up --build -d`	Не исполнялся: Docker CLI отсутствует
`requests/smoke.sh` против работающей системы	Не исполнялся здесь; инструкция и ожидания включены в проект

Перед сдачей требуется на целевом ПК выполнить README-шаги, приложить фактический вывод `docker compose ps`, `curl /healthz`, `./requests/smoke.sh` и проверку прав БД, затем опубликовать Git-репозиторий и заменить поле URL на реальную ссылку. Это единственная часть отчёта, которую нельзя достоверно завершить из среды без Docker, сетевой загрузки Go 1.25 и учётных данных удалённого Git-хостинга.

11. Карта кода и воспроизводимость

Файл	Содержание
`api/main.go`	HTTP routes/middleware, парольный verifier, sessions, rate limit, object auth, обращения к PostgreSQL, выдача, аудит.

Файл	Содержание
`api/main_test.go`	Unit-тесты UUID, JSON mass assignment, Argon2id.
`worker/main.go`	Outbox scanner, AMQP publisher confirmations, consumer, SMTP Mailpit, ACK/NACK.
`db/schema.sql`	Таблицы, FK/UNIQUE/CHECK, индексы, immutability и append-only triggers.
`db/roles.sql`, `db/03_worker_role.sh`	GRANT для API и worker; отсутствие DDL и лишних привилегий.
`docker-compose.yml`	Контейнеры, окружение, health checks, локальные порты и volumes.
`web/`	Клиент и конфигурация Nginx.
`requests/requests.http`	Коллекция запросов ко всем API-маршрутам: login/logout/recovery, приложения, выдача, документы, аудит и AI 501.
`requests/smoke.sh`	Проверки IDOR, mass assignment, Idempotency-Key, RBAC, выдачи, аудита, logout и прав БД.
`docs/change-log.md`	Подробный журнал расхождений CH-01…CH-16.
`docs/code-map.md`, `docs/validation.md`	Карта кода и ясное разделение выполненных static checks от ещё не выполненных runtime checks.

12. Выводы

Подготовлен целостный проект ЛР № 3 для варианта 1 на Go 1.25: API и worker, локальное окружение из семи контейнеров, PostgreSQL DDL с ограниченными сервисными ролями, веб-клиент, тестовые пользователи, smoke-скрипт и сопровождающая документация. Сохранены центральные проектные инварианты ЛР № 2: серверная объектная авторизация, неизменяемость документа после выдачи, атомарная фиксация выдачи и аудита, Outbox для асинхронного уведомления и запрет лишних привилегий приложения.

Результаты статической проверки и ограничения текущего исполнения зафиксированы явно. Для окончательной демонстрации на защите необходим запуск Docker Compose на целевом компьютере, выполнение unit/integration сценариев и публикация локальной Git-истории в удалённом репозитории. SHA-256 документа обеспечивает контроль целостности содержимого, но не заменяет электронную подпись и сам по себе не придаёт документу юридическую силу.

Приложение А. Команды для демонстрации на защите

```
cp .env.example .env  # заменить демонстрационные секреты
docker compose up --build -d
docker compose ps
curl -i http://localhost:8080/healthz
./requests/smoke.sh
docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
docker compose exec db psql -U postgres -d documents -c "SELECT rolname, rolsuper, rolcreatedb, rolcreaterole FROM pg_roles WHERE rolname IN ('documents_app','documents_worker');"
docker compose down  # остановка
docker compose down -v  # полный сброс демонстрационных данных
```

Приложение Б. Репозиторий

Локальная Git-история находится в каталоге проекта и включена в ZIP-комплект. URL удалённого репозитория не указан, потому что репозиторий не публиковался в GitHub/GitLab из этой среды. После создания удалённого репозитория вставьте реальную ссылку в эту строку и в README: URL: _____.